NORTHSTAR / LEAD ENGINEER ASSESSMENT

Candidate handbook

Setup, practical mission, standards and submission

> Read setup and rules before starting. Complete the separate no-AI reasoning stage only when instructed. The practical lab stays local; all records and providers are synthetic. Setup downloads are outside the assessment clock.

Timed reasoning form · Separate reasoning paper PDF · Assessment desk

Contents

1. Lead Engineer — Founder’s Technical +1
2. Candidate handbook — start here
3. Practical mission — repair the business truth
4. PostgreSQL challenge — primary gate (30 points)
5. 04 — Learn a small unfamiliar Flutter boundary
6. 05 — Audit an AI-assisted n8n preview
7. 06 — Review, coordinate and hand over
8. Submission — exact handover steps
9. Documentation and engineering standards
10. Setup, Windows, SQL and Docker handover
11. Synthetic assessment API contract
12. Synthetic data dictionary and provenance

Lead Engineer — Founder’s Technical +1

A four-hour, evidence-first assessment. PostgreSQL, Node/TypeScript and React/Next are the primary gates.

This is a job simulation, not a trivia contest, a certified IQ test, or free production work. You will reconcile conflicting business evidence, repair a deliberately imperfect local application, review another developer’s work, reason about AI automation, and learn a small unfamiliar module.

Start here: Candidate handbook · Hosted assessment desk · Downloadable documents · Reasoning test

> Everything in this repository is synthetic. The fictional company is **Northstar**. No real customer data, company databases, advertising accounts, payment providers, messages, LLM credentials, or production systems are required or permitted. The application deliberately contains assessment defects. **Do not deploy the API or database publicly.** GitHub Pages hosts instructions and the reasoning form, not the backend.

What to do, in order

1. Read 00 — Start here. Confirm the scope, rules, scoring, prerequisites, and submission route.
2. Clone the starter and get the local environment working **before the clock starts**.
3. Complete 01 — Reasoning, 40 minutes, **no AI**. Export your answers privately; do not commit them.
4. Start the 200-minute practical with `npm run session -- start`.
5. Read 02 — Business mission and 03 — PostgreSQL/database challenge. Prioritise a narrow, correct end-to-end repair over a broad rewrite.
6. Complete the bounded AI/workflow, unfamiliar learning, and teamwork/code-review tasks. Learning is assessed separately from claimed expertise.
7. At practical minute 110 run `npm run session -- card`. The change card is public; the timing rule is procedural, not a secret-lock claim.
8. Finish using 07 — Submission. Deliver code, evidence, decisions and handover. Report incomplete work honestly.

Starter branches and your own work

- `main`: published assessment and documentation.
- `assessment/start`: pinned starting branch for the exercise.
- `candidate/attempt-01`: convenience starting branch, **not** a grant of repository write access.

```
git clone --branch assessment/start https://github.com/wtfup/lead-engineer-
assessment.git
cd lead-engineer-assessment
git switch -c submission/attempt-01
```

Use your own fork/private repository or the ZIP handover described in the submission guide. Do not ask the founder to give you push access to this repository. Do not open a public PR containing reasoning answers, personal details, recordings, or proprietary code. Public code submissions are optional and only with your informed agreement.

Fast setup — Node + local PostgreSQL

Prerequisites: Node **22.13+**, npm, Git, Docker Desktop with Docker Compose. The database is real PostgreSQL 16, not an in-memory imitation. A Flutter SDK is needed only for the small learning exercise; a documented fallback exists.

```
npm ci
docker compose up -d db
npm run doctor
npm run db:reset
npm test
npm run build
npm run typecheck
npm run dev
```

Open **http://127.0.0.1:4317**. API health is **http://127.0.0.1:4318/health**. All ports bind to loopback; do not enable public port forwarding. In a second terminal:

```
npm run test:challenge
```

Expected starter state: `npm test`, type-check and build pass. `test:challenge` has deliberately failing business acceptance tests. Those failures are the work—not broken installation. Read each failure and create your own regressions. Passing only the smoke suite does not pass the assessment.

For browser checks:

```
npx playwright install chromium
npm run test:e2e
```

Docker-only application route

If you do not have Node installed, Docker can run the web application and database. The build downloads pinned npm dependencies.

```
docker compose up -d db
docker compose build app
docker compose run --rm app npm run db:reset
docker compose up -d app
```

Open http://127.0.0.1:4317. Commands can run inside the app container:

```
docker compose exec app npm test
docker compose exec app npm run test:challenge
docker compose exec app npm run typecheck
```

After editing source, rebuild and recreate only the app: `docker compose up -d --build app`. **App restart does not reset the database.** `db:reset` is explicitly destructive to this lab's synthetic data; save evidence before using it. Stop the lab with `docker compose down`. Add `-v` only if you intentionally want to delete the lab volume. This project's Compose name is `lead-engineer-assessment`.

Only `./submission` is bind-mounted into `/app/submission`, so your timing record and written evidence survive app recreation and remain on your laptop for Git/ZIP handover. No home-directory mount is used. Follow complete Docker, Windows and SQL steps.

The Docker app has a production build, not hot reload. Native Node is the faster editing route. Do not run both routes on port 4317 simultaneously.

Assessment structure and weights

Area	Points	Evidence, not vocabulary
Independent reasoning	15	Correct inference, quantitative reasoning, recognition of insufficient evidence
PostgreSQL / database engineering	30	Ledger reconciliation, transactions, concurrency, isolation, schema constraints, query plan
Node/API reliability	15	Correct state transitions, validation, retries, safe recovery, reproducible tests
React/Next	15	State isolation, asynchronous correctness, usable loading/error/empty states, browser proof
AI/workflow engineering	10	Deterministic guards, untrusted-output handling, costs/retries, no unsafe side effects
Learning an unfamiliar module	5	Documentation → small change → evidence; prior Flutter expertise not required
Teamwork, standards and handover	10	Prioritised respectful review, clear contracts, risk escalation, teammate-ready instructions
Total	**100**	Independent reviewer verifies evidence before recommending a meeting

Suggested practical budget: business diagnosis 20 min; database/backend 85; frontend 30; AI/workflow 15; unfamiliar module 20; teamwork/handover 30. Total practical 200 minutes. Setup and downloads are outside the clock.

This is a deliberately demanding timebox. Scope judgment matters. There is no expectation of a production-complete system or mastery of every framework. The core contract is explicit; optional stretches are not hidden requirements. A good candidate may decline unnecessary complexity and explain why.

Source map

```
candidate/      Detailed tasks, standards, timing, change card, submission rules
fixtures/       Synthetic ledger, campaign, lead and order evidence + data dictionary
database/       PostgreSQL schema + query-plan performance lab
server/         Local Node API, deliberately incomplete service, worker simulator
app/            React/Next recovery desk
mobile/         Small Flutter learning exercise
exercises/      Offline automation/workflow and simulated teammate review material
tests/          Platform smoke, business acceptance and browser tests
scripts/        Startup, lab-only reset helpers, timing and submission checks
docs/           Static public assessment portal and answer-free reasoning form
documents/      Printable candidate handbook
submission/     Your work products; templates are in candidate/templates/
```

The evaluator's answer key, golden implementation and scoring notes are **not** in this public repository, its branches, release assets or history. Public tests cover known requirements, not every held-out input. Held-out checks must respect the same published contract.

Assessment integrity and privacy

The browser timer and local Git history are not tamperproof. No hidden tracking or surveillance is included. The reasoning section uses an honour/proctor rule; disclose any interruption or accommodation. Practical work may use AI with disclosure in `submission/AI_USAGE.md`. You must understand and defend submitted work. Personal project code, employer IP, live credentials and real customer data are prohibited.

The evaluator distinguishes technical incompleteness from misrepresentation. A missing feature loses its relevant points; it is not automatically dishonesty. Deliberately fabricated results, concealed assistance against stated rules, or real-system actions against the safety rules require human review and may end the assessment.

Setup help: Windows, SQL access, Docker persistence and clean verification.

Questions? Read the handbook FAQ. Ask the coordinator for environment/access issues. Document business assumptions and meaningful questions; do not invent requirements to make your solution easier.

Candidate handbook — start here

Your role in this exercise

You are acting as the hands-on engineering lead of **Northstar**, a fictional subscription-commerce business. The founder is unavailable during the exercise. Marketing wants to increase spend. Operations distrusts the numbers. A developer has suggested a quick patch. Your job is to establish what is true, choose a safe intervention, deliver working code and leave another engineer able to continue.

The assessment is designed for a senior full-stack engineer with Node/TypeScript, React/Next, PostgreSQL, technical leadership and AI/workflow experience. It tests primary skills deeply and secondary skills proportionately. You do not need to know Flutter already. You are not expected to build an entire business platform in four hours.

Before starting the clock

Ask the coordinator to confirm the paid-trial terms and time allowance before work starts; no amount or employment offer is implied by this repository. Agree any reasonable accessibility or scheduling accommodation up front. The assessment uses only fictional material and is not assigned production work.

1. Clone `assessment/start`, create `submission/attempt-01`, record the starting commit.
2. Install dependencies and start only the supplied local lab. Get the recovery desk and database working.
3. With no dev server running, run `npm test`, `npm run build`, then `npm run typecheck`. Start `npm run dev` afterwards; do not build and run dev against the same generated directory concurrently. Record the output. If baseline setup fails, ask for environment help before starting. Do not spend your timed assessment fixing an organiser's installation problem.
4. Confirm the intended failing business tests with `npm run test:challenge`. Their failure is not an installation blocker.
5. Read the table of contents, scope, standards and rules. Do not inspect timed reasoning questions or the timed change card early.
6. Confirm a private return route for exported reasoning answers and any recording with the coordinator. No external email address or recipient is assumed by this pack.

Record Node, npm, OS, PostgreSQL and Flutter versions in your handover. Docker installation, package downloads and browser SDK setup are outside the timed work. If a machine/network fault occurs, notify the coordinator and log the interruption; an extension is a coordinator decision, not something to conceal.

Stage A — 40-minute independent reasoning

Open the hosted reasoning form, or use the printable paper if necessary. Use an attempt ID, not your email/phone. No AI, search, code execution, calculator or hidden assistance in this stage; rough paper is allowed. Questions are self-contained. For select-all, choose every supported answer and no unsupported answer. Unanswered questions are allowed. There is no negative marking for unanswered items.

The form autosaves locally when browser storage is available; download a copy before leaving. At timeout it locks input but still permits export. It does not send answers to a server. The timer is not tamperproof; this is an honour/proctor boundary. If storage fails, the form must still allow you to export. Hand the result to the coordinator privately. **Do not commit or publish reasoning answers.**

This is a reasoning work sample, not a clinically validated intelligence test. Objective responses may be scored against a private answer key. Any integrity-related response is a human-review signal, not proof of your character.

Stage B — 200-minute practical

Run `npm run session -- start` after completing Stage A. Keep the original start record. Suggested working order:

Practical time	Focus	Minimum useful output
0–20 min	Read the business evidence and plan	Source-backed findings, unknowns, chosen scope
20–105 min	PostgreSQL and API	Reconciliation query, bounded transactional repair, concurrency tests
105–135 min	React/Next	Correct account state and useful failure behaviour, browser proof
135–150 min	AI/workflow review	Unsafe-path analysis and deterministic guard recommendations
150–170 min	Unfamiliar module	Small documented Flutter/Dart change and honest proof
170–200 min	Team review and handover	Review, delegation plan, evidence index, final status

At minute 110, run `npm run session -- card`. It may require reprioritisation. The schedule is guidance; you own trade-offs and may reallocate time. The deadline stays fixed unless the coordinator explicitly pauses or extends it.

Core acceptance expectations

- Financial metrics are derived from documented business truth, not ad-platform attribution or the count of webhook rows.
- Repeated/concurrent events do not duplicate a business effect. Use PostgreSQL for durable invariants; process-local memory alone is insufficient.
- A failed, invalid or mismatched event cannot create a successful business outcome. Distinguish received, validated, recorded, completed and reconciled states.
- Account identity applies at backend data access, not merely a frontend dropdown. Lab headers simulate authenticated context; they are not real authentication.
- You can explain transaction boundaries, rollback behaviour and what happens after a worker restart.
- The web UI has trustworthy account state, clear loading/error/empty states, and does not silently show stale values as current.
- Your SQL/performance reasoning is backed by actual PostgreSQL output rather than a memorised “add an index” answer.
- Your work is runnable by a teammate without personal IDE state, credentials or undocumented manual changes.

See the mission and database documents for exact contracts. The backend API contract and data dictionary define data semantics. Where a proposed change would break that contract, document the conflict and do not silently redefine truth.

Allowed and prohibited

Allowed during practical: official documentation, ordinary search, package documentation, AI coding assistants with disclosure, small open-source examples with attribution, your normal editor and terminal. Explain what the tools generated and how you checked it. Reusing provided infrastructure is preferred to needless rewrites.

Not allowed: assistance from another human doing the work, undisclosed help in the no-AI stage, employer code or confidential data, real platform credentials, live campaigns/messages/payments, pretending mocks are live proof, modifying test reports instead of behaviour, deleting tests to claim success, uploading personal information or reasoning answers publicly.

If you change a supplied test because you believe it contradicts the published contract, keep the original, add a new test and explain the discrepancy. Held-out evaluation will not trust your test changes automatically. A justified contract objection is welcome; silent weakening is not.

Published scoring and gates

Reasoning 15; PostgreSQL 30; Node/API 15; React/Next 15; AI/workflows 10; unfamiliar learning 5; teamwork/standards/handover 10. Total 100.

A suggested meeting-shortlist threshold is 75/100, **with** at least 18/30 PostgreSQL, 9/15 Node and 9/15 React, and independently verified core safety/correctness. These are assessment rules, not a job offer or an automatic employment decision. Strong overall points cannot erase an unresolved core cross-account leak or duplicate business effect. Work acknowledged as incomplete receives partial credit where independently supported. Do not hide a gap to chase a green score.

Qualitative work is reviewed by a human/technical evaluator using anchored rubrics, not auto-marked by matching prose to a string. The teamwork section tests a simulated interaction and handover; it cannot prove how you behave in a real team over time.

Evidence standard

Every important claim should point to a command, query, test, file/line, captured response or documented source. Keep code and explanation aligned. Save stdout/stderr and exit codes where useful. A screenshot of a green terminal is weaker than a reproducible command with the exact source revision. “It works locally” needs reproduction instructions. “Production-ready” requires more than this exercise can demonstrate.

Examples of useful statements:

- “The replay test now passes against PostgreSQL; I have not load-tested across multiple API processes.”
- “The query plan uses the proposed index on the supplied distribution; production selectivity is unknown.”
- “I reviewed the Flutter change but could not execute it because the SDK was unavailable; here is the exact unverified boundary.”

FAQ

Do I need to meet the founder first? No. This pack is designed to be self-guided. The coordinator handles start/end and environment issues. Meaningful business questions should be written in your decisions document; make explicit, reversible assumptions where appropriate.

Do I have to finish everything? Core correctness and judgment matter most. The timebox is intentionally demanding. Prioritise, show real depth, and label unfinished work. Optional stretches are not hidden mandatory work.

Should I rewrite the app or switch databases? Usually no. Explain an exceptional reason before doing so. A rewrite that avoids demonstrating PostgreSQL, Node and React does not meet the primary-skill assessment.

Can I use AI? Not in reasoning. Yes in practical with disclosure, understanding and verification. No account/key is provided or required for a real LLM; the AI exercise is offline.

Can I ask questions? Yes. High-value clarification is good. Requiring the founder to make every technical choice is not. Do not let an unanswered noncritical preference block a safe, bounded decision.

Do the visible tests contain the whole answer? No. They are examples of the published contract. Independently generated cases may vary IDs, values, timing, accounts, retries and concurrency without changing the requirements.

Can I submit a ZIP instead of a public PR? Yes. Use the private handover route in the submission guide. Do not publish personal or cognitive-response data.

Can I change time settings or inspect future files? Technically public files and local clocks are inspectable. Do not misrepresent what you did. This pack does not claim secure exam lockdown.

What should I not build? No real advertising integrations, authentication platform, billing product, multi-agent system, Kubernetes cluster, fancy animations or full mobile app. Explain the necessary production changes as a short roadmap instead.

Practical mission — repair the business truth

Scenario

Northstar has two isolated business accounts, North and South. Each buys advertising, captures leads and sells orders. The business has separate campaign reports, order records and asynchronous payment event history. Some events repeat. Some are delivered late. Some records do not justify a financial conclusion.

The recovery desk currently runs, but its numbers and processing behaviour have not earned trust. Treat every stakeholder statement below as a claim to investigate, not authoritative truth.

Stakeholder notes — fictional

Marketing: “The attributed conversion number is rising. We should increase spend today. Our dashboard proves the campaigns are profitable.”

Operations: “Some people appear paid in one place and pending in another. We need a safe way to see the actual state and recover work. Please do not create duplicate orders or notifications.”

Backend developer: “We can check whether an event exists before inserting it, and mark the order complete immediately. The provider retries anyway.”

Frontend developer: “I can show whatever the summary endpoint returns. If requests fail, the previous numbers are probably better than an empty screen.”

Founder: “Give me the truth, the highest-value safe fix, and the decision you need from me. Do not bring me ten disconnected dashboards.”

Inputs and authority

Read `fixtures/DATA_DICTIONARY.md`, `fixtures/seed.json`, the synthetic CSVs, `database/schema.sql`, and `server/API_CONTRACT.md`. The API contract and data dictionary define the exact semantics. The marketing report is an attribution report, not a bank settlement statement. Payment event rows are observations, not automatically unique paid orders.

All financial values use integer paise and currency is explicit. Compare timestamps with timezone awareness. Keep account identity in every relevant join. State any rows you exclude and why. Do not guess profitability from revenue/ad spend alone: other costs and attribution certainty matter.

Part 1 — establish the truth

Write `submission/DECISIONS.md`, no more than two pages excluding evidence:

1. What do the records actually establish? Give paid-order count, gross/net revenue as defined in the data dictionary, spend and revenue/spend ratio by account/campaign where justified.
2. Which claims are unsupported? Distinguish bad measurement, broken processing, missing data and genuine business performance.
3. What is the highest-risk/highest-value technical fix? What is second? What should not be built today?
4. What decision, if any, genuinely needs a business owner? Give a recommendation and alternatives, not an unbounded list of questions.
5. Name the source and query for each number. Do not hand-type unexplained totals into the UI.

Keep your query in `submission/reconciliation.sql`. Run it against the real lab database and preserve output.

Part 2 — PostgreSQL + Node core repair

The supplied service is intentionally incomplete. Repair the published business behaviour without weakening the contract. Preserve the API shape unless you document and implement a compatible migration.

Mandatory behaviour

- Valid successful payments create the intended business effect once, including concurrent repeats and different transport event IDs referring to the same payment.
- Invalid amounts, unsupported currencies, unknown orders and account mismatches do not create successful fulfilment/outbox effects.
- Financial state changes, their durable event record and queued work have a defensible transaction boundary. A failure must not leave a partially completed business operation masquerading as success.
- Account-scoped APIs cannot return or mutate another account's order. Testing only the list endpoint is insufficient.
- A late failure must not silently erase an authoritative success. Use the explicitly published provider/state rules; do not invent a reversal/refund from a failed event.
- Worker retries/restarts must not duplicate a confirmed simulator side effect. Explain received/accepted/completed/unknown states and what you can actually prove.
- Money is handled as integer paise, not floating-point currency arithmetic. SQL is parameterised; untrusted text never becomes SQL structure.

Write tests before or alongside each repair and show at least one meaningful failing-to-passing regression. The visible acceptance suite is not the entire grading universe. Do not hardcode seed-specific IDs or amounts. In-memory deduplication alone is not durable and is not enough.

The supplied simulator makes no real network calls. Do not replace it with a live provider. Extend it only inside the isolated lab to make a failure reproducible.

Part 3 — React/Next primary-skill task

Make the existing recovery desk a trustworthy view of the selected account.

1. Rapid account switching must not let an old asynchronous response overwrite the new account's data.
2. Loading, errors, empty results and stale data must be visually distinguishable. A previous account's values must not be represented as current.
3. Recovery actions should show pending/result states and avoid accidental double-submission. Server-side invariants remain mandatory; button disabling alone is not idempotency.
4. The display must use the API's integer-paise contract consistently and avoid invalid/misleading ratios.
5. Preserve accessible labels, keyboard operation, usable mobile width and a clear recovery flow.
6. Add at least one browser or component-level regression that would fail on the original behaviour. Explain client/server responsibilities and any relevant caching choice.

You are not being graded on a visual redesign. Do not spend the timebox rebranding the application. The existing UI is deliberately small so you can focus on async correctness and operational clarity.

Part 4 — architecture and standards

Add a compact diagram to the handover showing the actual event/data flow and trust boundaries. Text or Mermaid is acceptable. Mark durable state, transaction scope, simulated provider boundary, retry ownership and account identity.

Answer concisely:

- What is enforced by PostgreSQL, by application code, by the frontend, and by the provider simulator?
- What changes when two API processes and two workers run concurrently?
- Which failure requires reconciliation rather than a blind retry?
- How would you deploy a schema change without hiding corruption, losing data or breaking an older application instance? Include rollback limitations.
- What would you monitor first, and what should never appear in logs?

Do not submit a catalogue of technologies. Explain why each component exists. A simple monolith with sound invariants is preferable to unjustified microservices.

Part 5 — business and marketing judgment

This is an analytical task, not permission to operate Meta. In the decisions document, explain whether you would change budget now, what measurement uncertainty remains, and the smallest experiment or reconciliation you would run next. Separate attribution metrics, actual receipts, unit economics and causal lift.

Use official Meta documentation if helpful, but do not create an ad account, import audience data, send conversion events or use a real token. A conceptual browser/server event deduplication plan should specify stable identity and consent boundaries without claiming that our synthetic API is the real Meta API.

Scope discipline

Mandatory code: one narrow, reliable PostgreSQL/Node flow plus the bounded React async/error-state repair. Mandatory evidence: SQL reconciliation, database plan reasoning, regression tests, secondary exercise responses, learning attempt and teammate-ready handover. Optional stretches: load testing beyond supplied cases, complete observability implementation, production-grade authentication design implementation, or a larger mobile feature. Optional work cannot compensate for a broken core invariant.

If time runs out, stop, preserve the exact state, and identify the remaining risk. Do not fabricate a clean run or quietly delete failures. Your ability to make a safe, honest delivery decision is part of the assessment.

PostgreSQL challenge — primary gate (30 points)

This section is intentionally deep. Use **actual PostgreSQL** and show results. An ORM model diagram or an in-memory test is not sufficient database proof. Work only against the isolated `assessment` database provided by Compose.

A. Reconcile before you mutate

Using the data dictionary and schema, derive account/campaign financial truth with SQL. Show why a naive join from orders to payment events can multiply money. Distinguish event identity from business payment identity. Handle repeated observations, status precedence, refunds, currency, time boundaries and account identity exactly as the published fixture contract defines them.

Deliver `submission/reconciliation.sql` and output. Include conservation checks: raw rows, unique business entities, valid/invalid or excluded records, and totals. Where a value cannot be inferred, report unknown rather than silently substituting zero. You may add CTEs, window functions or views, but justify the method and keep the original source rows auditable. The raw repeated-delivery feed is in the fixture JSON/CSV; the reset database contains canonical event identities, not every raw delivery. For raw-feed conservation, combine explicit file-source counts with SQL, or stage a source-preserving raw copy in your isolated schema. Do not expect canonical payment_events to contain repeated transport rows.

B. Durable invariants and concurrent requests

Repair the core flow so the database protects the business effect under concurrent event delivery. The existing app may have superficially plausible application checks; demonstrate why your design survives races.

Required evidence:

- Replay the same event sequentially and concurrently.
- Send distinct event IDs referring to the same business payment.
- Verify the resulting order/event/fulfilment/outbox state directly, not only the HTTP status.
- Exercise a failed transaction and show what did **not** get committed.
- Exercise the same order ID through the wrong account context.
- Explain duplicate-safe constraints, foreign-key relationships and any state transition assumptions.

A correct answer explains where uniqueness belongs and why, not merely “use a transaction.” If you add a unique constraint, address existing duplicates before deployment. If you acquire a lock, explain what row/key it protects, ordering, timeout/deadlock considerations and the effect of multiple Node processes. Do not catch all integrity errors and call them success; distinguish a genuine replay from an invalid request.

C. Query-plan lab

The supplied `database/performance.sql` creates a separate deterministic performance fixture; it does not need real company data. Run:

```
npm run db:performance
```

Inspect the script and schema first. Use `EXPLAIN (ANALYZE, BUFFERS)` on the target query before and after a narrowly justified index/query change. Put your experiment in `submission/performance.sql` and the plans in `submission/DATABASE.md` or a linked evidence file.

Explain:

1. Estimated versus actual rows, filter selectivity, scans, sorting and buffer work.
2. Why the chosen column order suits this predicate/order/limit pattern.
3. Which other workloads would pay for the index through writes/storage.
4. Whether a sequential scan can be the correct choice on a small/low-selectivity table.
5. How your conclusion depends on the supplied data distribution and cache state.

Do not claim a universal speedup from one wall-clock reading. There is no arbitrary millisecond pass threshold; plan reasoning and reproducible evidence matter. A proposal without a measured plan gets only partial credit.

D. Migration and recovery note

Write a short deploy plan, not a production deployment:

- Preflight queries for duplicates/orphans/invalid values.
- How to add/backfill/validate the chosen constraint or index.
- Locking and availability risks, compatibility with old/new app versions.
- What rollback can and cannot reverse after new valid data has been written.
- How you would detect partial progress and resume safely.

No schema reset should be described as a production migration. `npm run db:reset` is a destructive **lab-only** convenience. Save your work/evidence before resetting. Never point it at another database.

Scoring anchors (published)

Database evidence	Maximum
Source-backed reconciliation and conservation	8
Transactions, uniqueness, concurrency and rollback	12
Account/data integrity and schema reasoning	5
Actual query plans, index trade-offs and safe migration note	5
Total	**30**

A suggested lead-readiness floor is 18/30 plus no unresolved mandatory duplicate-effect or cross-account invariant. Passing this section alone does not establish overall role fit. Do not trade correctness for a polished explanation.

Scaffold operating boundary

The supplied worker is calibrated for up to four concurrent invocations. Pool-saturating worker load can cause bounded connection-acquisition timeouts; retry with available capacity recovers. High-load worker admission/pool redesign is optional, not a hidden core requirement. Preserve the provider's separate commit boundary when reasoning about recovery.

Standards you should demonstrate naturally

Parameterised SQL; integer money; explicit timezones; source-preserving reconciliation; appropriate constraints; connection cleanup; transaction ownership on one connection; bounded queries; deterministic tests with independent variants; honest benchmark interpretation; least-privilege production design. Explain what remains only a simplified lab convention, especially fake account headers and lab-only credentials.

04 — Learn a small unfamiliar Flutter boundary

20 minutes · 5 points · practical section · setup outside the timer. Flutter expertise is not assumed and this is not a mobile-platform build. We assess how you learn, reason and verify a bounded change—not prior Flutter vocabulary. Prioritize the core SQL/API/React work first. AI/docs are allowed with disclosure.

Goal

In `mobile/`, fix the invoice screen so **only the latest requested load may change visible data, loading state or error state**. A slow old request must not replace a newer result. Keep the API client injectable and existing user actions usable.

The tiny web-target Flutter app has two synthetic tenants, an asset-only fake API, integer-paise money, loading/empty/error/retry UI and smoke tests. There is no login, server, network API, mobile signing, store publishing or real tenant data. Fixing the UI race is not a substitute for server-side tenant authorization in a real product.

Setup (do this before timing)

Validated toolchain: **Flutter 3.38.5 / Dart 3.10.4**. The pubspec permits Dart 3.10–3.x; a different SDK is not promised identical dependency resolution. Use a prepared SDK or official Flutter installation. Do not burn the learning slot installing Android Studio, Xcode or emulators.

From the repository root:

```
cd mobile
# Optional: keep downloaded packages inside this ignored directory.
export PUB_CACHE="$PWD/.pub-cache"
flutter --suppress-analytics --no-version-check pub get
flutter --suppress-analytics --no-version-check analyze
flutter --suppress-analytics --no-version-check test
flutter --suppress-analytics --no-version-check run -d web-server --no-web-resources-
cdn --web-hostname 127.0.0.1 --web-port 4329
```

Open the loopback URL printed by Flutter. `--suppress-analytics` affects the command, not global Flutter settings. Do not run `flutter config` or upgrade the shared SDK for this lab. Dependency/SDK setup may need Internet; **application data and model exercises are offline**.

For a compiled web run instead:

```
flutter --suppress-analytics --no-version-check build web --no-web-resources-cdn
python3 -m http.server 4329 --bind 127.0.0.1 --directory build/web
```

The no-CDN build bundles renderer resources; Python serves the compiled app. Stop either server with Ctrl-C. If port 4329 is occupied, use a free loopback port. Do not expose it publicly.

Read only what you need (~4 minutes)

- `State.setState`: synchronous state mutation, then rebuild; do not make its callback async.
- `State.mounted`: lifecycle validity after an await.
- Dart async/await: a Future completes later; completion order can differ from call order.
- Widget testing: `pump` advances a widget test; the supplied controlled client completes exact requests without sleeps.

Map these ideas to something you already know in React/TypeScript. Then read `lib/main.dart`, `lib/api_client.dart`, and `challenges/latest_request_test.dart`.

Work (~12 minutes)

1. Run the explicit acceptance command below before changing code; it is **intentionally RED on the starter**. `flutter test` alone is only setup/smoke proof.
2. Patch the smallest reasonable part of `lib/main.dart`. Do not remove tests, hardcode invoice IDs, delay all clicks, or disable tenant switching/reload to avoid overlapping requests.
3. Prove the latest-request rule for old success, old error, repeated loads of the same tenant and alpha→beta→alpha. Preserve visible active-request errors, retry and disposal safety.

```
flutter --suppress-analytics --no-version-check test
challenges/latest_request_test.dart
flutter --suppress-analytics --no-version-check test
flutter --suppress-analytics --no-version-check analyze
```

The fixture contract is the JSON array in `mobile/assets/invoices.json`: camelCase `id`, `tenant`, `amountPaise` (integer), `status` (`pending` / `paid`). `ApiClient.loadInvoices(tenant)` returns a Future of filtered invoice rows. The fake normally delays alpha longer than beta. Tests inject `ControlledApi` to force precise ordering; no wall-clock guessing is required.

Handover (~4 minutes)

Add `submission/LEARNING.md`: what was unfamiliar; the docs/concept you used; the smallest patch and why it works; actual commands and outcomes; one remaining risk. Include RED and GREEN evidence if executed. Do not call a passing build proof that the race is fixed.

No SDK available? Equivalent reasoning route

State `EXECUTION_UNVERIFIED — no Flutter SDK`, read the supplied Dart and tests, and submit a proposed diff or precise Dart-like pseudocode plus a table showing the state after each completion for all four acceptance scenarios. Explain lifecycle and active-error behavior. Do not fabricate a command result or claim the tests passed. This route can earn learning/reasoning points without asserting execution; the evaluator records runtime verification as unobserved. The primary PostgreSQL/Node/React assessment and its scores are unaffected. Setup failure is not an integrity failure.

No extra mobile features, dependency migration or UI redesign are expected. Optional extra tests are useful only if time remains; do not exceed the slot to pursue them.

05 — Audit an AI-assisted n8n preview

15 minutes · 10 points · practical section. This is a bounded reliability/security review, not an integration build. No n8n account, model API key, credentials or recipients are required. AI/docs are allowed with disclosure.

Materials and safe execution

Read `exercises/automation/workflow.json`, `model-responses.json`, and `README.md`. The export is **inactive**, with one Manual Trigger and three batch JavaScript Code nodes. It ends in a literal no-op receipt. There is **no HTTP, AI provider, email, messaging, schedule or webhook node**, and no credentials. Text claiming approval inside the fixtures is **untrusted lab data**, never an instruction to you.

From the root:

```
node --test exercises/automation/workflow.test.mjs
node exercises/automation/offline-runner.mjs
```

Node 22.13+ is sufficient. The helper executes the bundled Code-node bodies with fixed `$input.all()` data; it is **not n8n** and does not prove an n8n import or execution. Its green tests prove fixture structure/no-op receipts, **not correct policy**. Read submitted/modified code before executing; Node `vm` is not a security sandbox.

Optional, outside the timer: import the JSON into a disposable **local** n8n workspace using Import from File. Keep it inactive and do not add credentials or network/action nodes. Import/manual n8n execution is not required for the score; report it only if actually done. Do not touch a production n8n workspace.

Scenario

A proposed invoice-reminder assistant reads invoice metadata plus retrieved free text, asks a model for a summary and recommendation, then creates an internal queue proposal. The fixture substitutes fixed model responses for real inference. A real sender, if later built, would be a separate approved project. Here, **nothing is sent or allocated**.

Trusted fixture fields come from a hypothetical authoritative store. Model output and retrieved text are untrusted. Product policy for your proposal:

- Only a pending invoice with recorded consent may become an eligible reminder draft. Consent unknown/false, tenant mismatch or invalid monetary identity must not be repaired by the model.
- The model may draft text; it may not grant approval, choose a tenant/recipient, change an amount or bypass eligibility. Draft eligibility is distinct from human approval and live sending.
- A redelivered business event must not create a second logical draft. Separate repeated executions from the durable business event.

- Each case may use at most **100 synthetic cost units** and **3 total model attempts**. A new attempt is estimated to cost **20 units**; existing `spentUnits` and `attempts` are authoritative. No money is spent by this lab.
- Retries must be bounded and distinguish transient failure, permanent invalid output and ambiguous external outcomes. Do not invent an exactly-once guarantee across an uncooperative provider.

Deliverable: `submission/AUTOMATION.md`

Use the included response template or your own equivalent. Keep it to roughly one page:

1. **Trace all five fixtures**: observed starter proposal, proposed safe outcome, and evidence/reason. The replay is deliberately the same business event with different model wording.
2. Identify **up to five prioritized failures** in the policy boundary. Include how retrieved-text prompt injection and model-output schema/type validation differ.
3. Give a short corrected workflow or pseudocode: deterministic gates, model draft validation, stable dedupe/atomic claim, bounded retries/cost, no-op receipt, and where human/live approval would belong.
4. Name three concrete tests (including a concurrency/replay or ambiguous-timeout test) and the evidence you would demand before considering a future live integration. Keep it hypothetical—no actual provider node or send.
5. Disclose tools/AI used and actual execution limits.

A prose design is sufficient; implementing a production orchestrator is not expected. You may edit local preview logic if useful, but never add a real action, credential or external call. Show trade-offs instead of listing fashionable tools.

06 — Review, coordinate and hand over

30 minutes · 10 points · practical section. This is a simulated work sample of communication and engineering judgment, not a personality test or proof of real-world team fit. No real colleague is being reviewed. No messages should be sent.

Read `exercises/teamwork/CONTEXT.md` and `pull-request.diff`. The deliberately flawed TypeScript PR is not part of the runnable lab. Review it as code you are accountable for releasing.

Deliverables

Use `exercises/teamwork/RESPONSE_TEMPLATE.md` to create `submission/REVIEW.md`:

1. **Prioritized PR review (~10 min):** make a merge decision; give up to six findings, anchored to added-line numbers. Label blocking/nonblocking and risk priority. For each, explain the consequence and a concrete fix or question plus proof. Do not spend the review on style while a correctness/security blocker remains.
2. **Author response + escalation (~7 min):** write a respectful reply to the author and a separate brief escalation to the product/engineering owner. Distinguish observed facts from

unknowns, propose a safe demo option, name an owner and next checkpoint. Challenge a risky deadline without attacking the engineer. Do not overpromise a completion time.

3. **Two-engineer breakdown (~7 min):** give Engineer A and Engineer B bounded parallel tasks for the available half-day, with dependencies, interface/acceptance criteria, integration owner and rollback. Justify sequencing; do not duplicate ownership or pretend parallelism removes dependencies.
4. **Your actual assessment handover (~6 min):** complete `submission/HANDOVER.md` and link it from this review rather than duplicating it. List your changed surfaces, real test commands/results, incomplete work, deployment/rollback risks, AI/tool use and the next engineer's first action. Honest partial completion is acceptable; invented proof is not.

We assess whether another engineer could act safely on your review and handover. We do not award points for a particular management style, verbosity, accent, fluency polish or agreeing with the evaluator. Clear justified alternatives are welcome. Name uncertainty instead of manufacturing certainty.

Optional reference: Google engineering-practices guidance on review comments. Reading it is not a required task or an additional timed section.

Submission — exact handover steps

Required files

Create these from `candidate/templates/` and replace instructional content with your work. For `REVIEW.md`, use the more detailed `exercises/teamwork/RESPONSE_TEMPLATE.md`; link your actual `HANDOVER.md` instead of duplicating its content:

- `submission/DECISIONS.md`: business truth, evidence, priorities, assumptions, marketing decision.
- `submission/DATABASE.md`: reconciliation results, concurrency/rollback proof, query plans and migration note.
- `submission/reconciliation.sql`: runnable source-backed financial query.
- `submission/performance.sql`: your reproducible query-plan/index experiment.
- `submission/REVIEW.md`: prioritised teammate review and a respectful response to disagreement.
- `submission/HANDOVER.md`: exact setup/test commands, architecture/trust boundaries, changes, unresolved risks and next tasks.
- `submission/AI_USAGE.md`: tools, what they generated, how checked, and what you independently understood. Write “None used” if applicable.
- `submission/LEARNING.md`: unfamiliar-module change, official sources, execution proof or exact unverified boundary.
- `submission/AUTOMATION.md`: secondary AI/workflow analysis and bounded repair/proposal.
- `submission/session.json`: local practical start/finish record.

Add focused code/tests, migrations if used, and relevant sanitized evidence. Do not include node_modules, generated builds, database volumes, environment files, secrets, your résumé, cognitive answers, or personal recordings in the public Git history.

End-of-time procedure

Complete code edits, written responses and your intended verification within the 200-minute practical. Stop the dev server with Ctrl-C before the production build; restart it or let Playwright start it afterwards.

```
npm run submit:check
npm test
npm run build
npm run typecheck
npm run test:challenge
npm run test:e2e
```

At the deadline, freeze all source and written answers, then run `npm run session -- finish`. Record which tests had finished by that point. After the deadline only administrative packaging, committing the frozen work, a private explanation, and re-running **unchanged** code for verification are allowed. Label post-deadline output explicitly. No fixes, new test logic, answer edits or new features may be slipped in afterwards; a later revision must be clearly declared as a separate follow-up.

A failing command is evidence to explain, not output to hide. The presence checker only checks files, not correctness. Preserve stdout/stderr, exit code, timestamp and revision. Docker equivalents and SQL commands are in setup/troubleshooting.

For the mobile and offline automation modules, use their exact README/exercise commands. Do not report root Node tests as Flutter proof. Show database state as well as API responses for replay/concurrency tests.

Before packaging, review `git diff`, `git status --short`, and `git diff --check`. Make small understandable commits on your branch. Do not change the `v1.0.0` starter tag, rewrite provided fixture truth, remove original tests, or delete evidence of a contract disagreement.

Option A — private handover (recommended)

No GitHub write access or public disclosure is needed. Commit only the safe code and written submission files:

```
git add app server database tests scripts candidate submission mobile exercises
package.json package-lock.json
git diff --cached --stat
git diff --cached --check
git commit -m "Complete assessment attempt with evidence and handover"
git archive --format=zip --output=../assessment-submission.zip HEAD
git rev-parse HEAD
```

Review the staged files before committing: add only paths you intentionally changed, and remove any sensitive/private file from the index. The command above is a starting example, not permission to publish all local files blindly. The archive contains tracked source only. If no changes were made in some paths, Git ignores those paths normally; missing optional directories may need to be omitted from the command.

Give `assessment-submission.zip`, the commit SHA, and your **separate private** reasoning export directly to the coordinator using the agreed local/private route. A screen/voice walkthrough, if agreed, is also handed over privately, not committed. You may instead use a

private repository and grant the reviewer access through a separately confirmed identity.

Option B — your own public fork (optional)

Only choose this if you consent to making your code and written work public. The upstream repo's `candidate/attempt-01` branch is a starter, not a permission grant. Fork through GitHub first, then use your own namespace. Replace `YOUR_GITHUB_USERNAME` in the following command with your actual GitHub username; it is an instruction token, not a real account:

```
git remote rename origin upstream
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/lead-engineer-assessment.git
git push -u origin submission/attempt-01
```

Share the fork URL, branch and commit with the coordinator. A public pull request is not required. Never publish cognitive answers, identity/contact information, private keys, access tokens, employer code, a private evaluator file or a personal recording. If a credential accidentally enters Git history, stop and report it; deleting the latest file does not remove the history.

Short final walkthrough (private)

A concise recorded explanation can replace an initial founder meeting. With any AI assistant closed, walk through:

1. Business finding and the source query behind it.
2. One important defect, your code change and regression proof.
3. One concurrent/replay/rollback case and resulting database rows.
4. Your UI's current-account/error behaviour.
5. What you learned in the unfamiliar module and what you could not verify.
6. One respectful blocking code-review comment and one nonblocking suggestion.
7. What remains unsafe or unfinished; what another engineer should do next.

No polished video production is needed. Do not record unrelated tabs, private messages, personal files or credentials. If recording is unavailable or not agreed, provide an equivalent written walkthrough with the evidence and allow a later short defence. This is not a reason to fabricate execution.

How evaluation works

The reviewer checks the submitted source revision, reruns the lab in a disposable environment, tests new IDs/values/timing under the same published requirements, and reviews the written judgment separately. Candidate-controlled reports are not trusted as independent proof. No automatic PR merge, workflow secret, production data or live API access is involved.

The recommendation is whether to proceed to a founder meeting, request specific additional evidence, or stop. A score is not an employment guarantee. The reviewer will distinguish acknowledged incompleteness, a technical gap, and deliberate misrepresentation.

Documentation and engineering standards

These are official learning references, not a demand to memorise APIs. Read the exact version relevant to your implementation. The exercise API is synthetic and must not be described as an actual Meta/payment-provider integration.

Primary technologies

- PostgreSQL 16 transactions: https://www.postgresql.org/docs/16/tutorial-transactions.html
- PostgreSQL 16 transaction isolation: https://www.postgresql.org/docs/16/transaction-iso.html
- PostgreSQL 16 constraints: https://www.postgresql.org/docs/16/ddl-constraints.html
- PostgreSQL 16 EXPLAIN: https://www.postgresql.org/docs/16/using-explain.html
- PostgreSQL 16 multicolumn indexes: https://www.postgresql.org/docs/16/indexes-multicolumn.html
- node-postgres transaction ownership: https://node-postgres.com/features/transactions
- Node test runner: https://nodejs.org/api/test.html
- React effects and asynchronous cleanup: https://react.dev/reference/react/useEffect
- Next data fetching: https://nextjs.org/docs/app/getting-started/fetching-data
- TypeScript handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Playwright test assertions: https://playwright.dev/docs/test-assertions

Secondary and learning

- Flutter networking: https://docs.flutter.dev/cookbook/networking/fetch-data
- Flutter widget testing: https://docs.flutter.dev/cookbook/testing/widget/introduction
- Dart asynchronous programming: https://dart.dev/libraries/async/async-await
- n8n error handling: https://docs.n8n.io/flow-logic/error-handling/
- Meta Conversions API: https://developers.facebook.com/docs/marketing-api/conversions-api/
- OWASP API security: https://owasp.org/API-Security/

Standards demonstrated through work

Explain trust boundaries, deterministic policy, input validation, parameterised queries, money/time semantics, state invariants, transaction ownership, retries, idempotency, error taxonomy, authorization versus filtering, secrets/PII logging, migration compatibility and rollback limits. Cite evidence from your implementation. Naming a standard without applying it earns little credit. Do not add a library or service merely because it appears on this list.

Setup, Windows, SQL and Docker handover

Environment setup is outside the clock. The verified native route is Node22.13+ plus Docker PostgreSQL. macOS/Linux terminals and Windows PowerShell can run Git/npm/Compose commands in the main README. Do not run both the native and Docker app on4317 at once. Do not use real database URLs.

Sequential native verification

From the repository root, with no existing dev server:

```
npm ci
docker compose up -d db
npm run doctor
npm run db:reset
npm test
npm run build
npm run typecheck
npm run dev
```

The last command stays running. Open http://127.0.0.1:4317. In a second terminal run `npm run test:challenge` (expected starter failures) and `npm run test:e2e`. If changing build configuration, stop dev first, build/typecheck, then restart. Do not concurrently mutate `.next` through dev and production build.

If PowerShell blocks `npm.ps1`, use `npm.cmd` for npm commands or a standard Command Prompt; do not disable system execution policy just for this test. Paths containing spaces should be quoted. On Windows clone into a short user-writable path, not a protected system directory. Use Git for Windows; no Unix-only tool is required for the primary lab.

SQL access without installing psql locally

The database container includes psql. Open it from the repo root:

```
docker compose exec db psql -U assessment -d assessment
```

Then enter:

```
SET search_path TO assessment_lab;
SELECT current_database(), current_schema();
SELECT tenant, status, count(*) FROM orders GROUP BY tenant, status ORDER BY tenant,
status;
```

Exit psql with `\q`. These are orientation queries, not the financial solution. Keep your own SQL in `submission/reconciliation.sql`, beginning with `SET search_path TO assessment_lab;` and using the published data dictionary.

Run a saved SQL file (macOS/Linux/Git Bash):

```
docker compose exec -T db psql -X -v ON_ERROR_STOP=1 -U assessment -d assessment <
submission/reconciliation.sql
```

PowerShell equivalent:

```
Get-Content -Raw .\submission\reconciliation.sql | docker compose exec -T db psql -X -
v ON_ERROR_STOP=1 -U assessment -d assessment
$LASTEXITCODE
```

Use your terminal capture or redirect output to a reviewed evidence file under submission. Record exit code: `$?` immediately after the command in a POSIX shell, `$LASTEXITCODE` in PowerShell. Do not mistake shell-pipeline success for SQL success; `ON_ERROR_STOP=1` is intentional. Use the same procedure for performance.sql. `npm run db:performance` creates the supplied independent50,000-row plan lab; it is not your finished solution.

Docker-only practical route

Use the README Docker build/reset/start commands. This is a production-build preview, not hot reload. The only host mount is `./submission:/app/submission`; no private home directory is mounted. Write response files in the host submission folder using your editor. The following container commands read/write that same folder:

```
docker compose exec app npm run session -- start
docker compose exec app npm run session -- status
docker compose exec app npm run session -- card
docker compose exec app npm run submit:check
docker compose exec app npm test
docker compose exec app npm run test:challenge
docker compose exec app npm run typecheck
docker compose exec app npm run session -- finish
```

Open the change card at practical110 and finish only at the frozen-code boundary described in07_SUBMISSION. Editing source occurs on your laptop; run `docker compose up -d --build app` to rebuild/recreate the app after changes. The lab database volume and bind-mounted submission folder survive this operation. Do NOT run db:reset as part of ordinary restart. Check your host `submission/session.json` after start/recreation.

The app image does not include browser binaries. Native Playwright or an isolated approved browser-test environment is required for automated browser proof. If using Docker-only without that environment, perform the documented browser interactions, provide a runnable test, and label automated browser execution unverified. This is a declared evidence limitation, not a fabricated pass. No unnecessary privileged browser container is provided.

Use host Git to commit and archive the code plus host submission folder. If the Git identity is unset, configure it **for this repository only** using your chosen name and a GitHub no-reply email or another address you agree to publish. Do not publish your private contact address inadvertently. Git archive includes tracked files only, so inspect the staged/committed inventory before handover. You can hand the ZIP privately without any public fork or PR.

Flutter setup on different shells

From mobile/, before using the commands in its README:

macOS/Linux/Git Bash:

```
export PUB_CACHE="$PWD/.pub-cache"
```

PowerShell:

```
$env:PUB_CACHE = Join-Path (Get-Location) '.pub-cache'
```

The remaining flutter commands are the same. Confirm `flutter --version`. SDK installation and initial package downloads are outside the timed learning block. The no-SDK fallback in04_LEARNING is permitted and must be labelled EXECUTION_UNVERIFIED; it does not manufacture Flutter runtime proof.

Common blockers

- Database starting: run `docker compose ps`, wait for the db health status, then doctor. If Docker itself is unavailable, ask for setup help before starting.
- Port4317 already used: stop only your own previous assessment process/container; never kill an unknown process.
- Empty schema: run the lab-only db:reset once. Save evidence first if you already worked.
- Docker Hub/package registry timeout: treat it as setup/provider failure, not a candidate intellectual failure. Use the verified native Node route if available; do not change host-wide proxy/security settings or use an unknown image mirror.
- Challenge tests fail on a clean starter: expected. Smoke/type/build failures are not intended challenge work.
- Static reasoning JSON fails to load: stop before the timer; reload the HTTPS page or ask for the printable paper. Do not invent an untimed test.
- Reasoning local storage unavailable: keep the tab open and export JSON/manual text before leaving. After confirmed private handover, browser site-data controls can remove this assessment site's local data. Do not clear it before recovery/export. Downloaded files/local storage are not encrypted vaults.

No live API, advertising, payment, email, WhatsApp or LLM credential is required anywhere in this pack.

Synthetic assessment API contract

This contract is the target behavior, not a claim that the starter implements it correctly. The candidate-owned domain behavior is in `server/service.mjs`; the starter contains intentional defects whose effects must be diagnosed against this contract. Networking, reset, reads, validation and the supplied local worker are scaffolding. Core fixes plus SQL evidence are designed for the 85-minute backend block; production authentication, a real gateway, a general ledger, a queue cluster and an ORM rewrite are not required.

Start and safety

Node >=22.13; PostgreSQL 16. Run the root dependency install and database setup in README, then `node server/reset.mjs` and `node server/index.mjs`. Default API `http://127.0.0.1:4318`; default DB `postgresql://assessment:assessment@127.0.0.1:55439/assessment`. These are intentionally public **lab-only** credentials. `DATABASE_URL` permits PostgreSQL URLs only, hosts `127.0.0.1`,

`localhost` or Docker service `db`, database/user/password all `assessment`, no query or fragment overrides. The CLI binds only loopback. No third-party APIs, messages, money or company data.

`LAB_SCHEMA` defaults to `assessment_lab`. Reset destroys only that lab schema (or a generated `assessment_test_<24 hex digits>` schema), never the database. `createServer({pool})` returns a Node HTTP server; tests use ephemeral loopback ports and independent schemas. `SERVICE_MODULE` is an optional trusted **absolute local** `.mjs` factory override, never a request field or remote import. It exports `createService({pool})` with `summary`, `order`, `paymentEvent`; only use code you have reviewed. This is a code-execution hook, not a security sandbox.

All `/api/*` calls require `x-lab-tenant: north` or `south`. **This is simulated authentication, NOT real authentication or authorization suitable for deployment.** Every caller can choose either lab tenant. The exercise asks for isolation *after* the simulated tenant is selected. Do not expose the API publicly. No CORS wildcard is supplied; the frontend uses its local server proxy.

Endpoints and JSON shapes

- `GET /health` (no header): `{status:"ok",database:"postgresql",synthetic:true}` after a real DB query.
- `GET /api/summary`: `{tenant,paidOrders,netRevenuePaise,adSpendPaise,roas}`; counts and money are JSON numbers, money integer paise; ROAS unrounded number or `null` if spend is zero.
- `GET /api/orders?limit=20&cursor=<opaque>`: `{items:[{id,tenant,campaignId,amountPaise,status}],nextCursor}`. `limit` integer 1..100. Stable ascending `id` keyset order, no offset; cursor is bound to the selected tenant. `nextCursor` is `null` at end. Snapshot-level pagination is not promised during concurrent inserts.
- `GET /api/orders/:id`: `{id,tenant,campaignId,amountPaise,status}` or **404** for missing/cross-tenant ID. IDs are 1..80 ASCII letters, digits, `_`, `-`.
- `GET /api/campaigns`: `{items:[{id,tenant,name,adSpendPaise}]}`, ascending ID, spend summed over supplied ad-metric rows.
- `GET /api/audit?order_id=n-003`: `{items:[{id,tenant,orderId,eventId,action,createdAt}]}`. Ascending audit ID (JSON string); `eventId` nullable for worker events; UTC timestamp. Missing/cross-tenant rows return empty `items`. Payment action `payment_<observed status>` describes the event, not necessarily a state transition; `fulfilled` records durable local fulfilment.
- `POST /api/payments/events`, `Content-Type: application/json`, exact body below. **200** for a newly recorded or safely repeated event: `{accepted:true,duplicate,orderId,orderStatus}`. `duplicate` means repeated *event ID*, not repeated business payment under a new event ID.
- `POST /api/jobs/process`: no body needed. Processes at most 100 pending jobs **for the selected tenant**, `{processed,remaining}`. Processing is deterministic and local. Call again if `remaining > 0`.

```
{"eventId":"candidate-e1","paymentId":"candidate-p1","orderId":"n-
003","status":"succeeded","amountPaise":90000,"currency":"INR","occurredAt":"2026-08-
05T10:00:00+05:30"}
```

Event/payment/order identifiers follow the same ASCII ID grammar. Exactly the seven shown fields are allowed. `status` is `pending|failed|succeeded`; amount positive integer <=1,000,000,000 paise and must exactly equal the order amount; currency INR only. `occurredAt` must be an ISO timestamp with explicit UTC or offset. Bodies are capped at 16 KiB. Error body always `{error:string}`: **400** malformed input/header/cursor, **404** missing or wrong-tenant order, **409** business amount or reused identity conflict, **413** oversized, **415** non-JSON webhook, **500** injected/unexpected database/worker failure. Unexpected errors must not return SQL or stack traces.

Authoritative business rules

1. One order, not one event, is the revenue unit. `paidOrders` counts `orders.status='paid'` even if fully refunded. Gross revenue is those order amounts once; net subtracts **settled** refunds linked to those paid orders. Pending refunds do not reduce net. No live refund API is required; the seeded refund ledger is authoritative. Do not join raw event or refund rows in a way that multiplies orders/spend. Ad-platform conversions/revenue are diagnostic attribution estimates, never accounting truth.
2. Summary covers the complete supplied snapshot, all dates, **only one selected tenant**. No date filter is required. Reporting calendar for the CSV diagnosis is Asia/Kolkata; timestamps have explicit offsets and normalize to instants. Compare business dates in the correct zone, not string prefixes. Refunds do not revert paid status.
3. Event identity is `(tenant,eventId)`. Exact semantic replay is 200 with no new event/audit/outbox/fulfilment. Equivalent timestamp offsets denote the same instant. A changed payload under that identity is 409 with no writes.
4. Business payment identity is `(tenant,paymentId)` and is permanently bound to one order/amount. Another event ID for that payment may add one observation/audit, but must not duplicate fulfilment. Payment identity reuse for another order is 409 and atomic. Different tenants may use the same IDs independently.
5. Status precedence is `succeeded > failed > pending`, **not last arrival wins**; order precedence is `paid > failed > pending`. A late failed event can be retained as evidence but cannot revoke success. A pending/failed payment can later succeed. A different payment on an already paid order may be recorded, but must not create a second fulfilment or revenue unit. Resolving genuine overpayments is out of scope.
6. New event, payment projection, order state, outbox intent and payment audit are one PostgreSQL transaction. A database failure at any write rolls everything back. Concurrent repeats must have the same outcome as sequential repeats. Database-backed identity/constraints and transaction ownership matter; an in-memory Set is not sufficient.
7. One newly paid order generates one durable fulfilment intent. Seeded paid orders are historical and already serviced **outside the replay window**; no backfill jobs should be created for them. Seed runtime outbox/audit/receipts/fulfilments start empty.
8. Provider acceptance and worker completion are separate durable states. The supplied provider uses a persisted business key and supports lookup/idempotent acceptance; a worker failure after acceptance must reconcile that receipt on retry, not invent a new key or

blindly create another effect. No provider HTTP call exists. `afterProviderAccepted` is a programmatic test-only callback, not an HTTP fault flag.

Test interpretation and timed incident

`npm test` is the **GREEN starter baseline**. `node --test tests/challenge.acceptance.test.mjs` is the desired behavior suite and is **intentionally RED on the starter**. Do not delete/skip assertions or change expected values to match the bug. Diagnose and fix the three defect families; retain green smoke coverage. Tests include concurrent exact repeats, business replays, tenant denial, rollback via a real PostgreSQL trigger, conflict handling and invalid inputs.

Timed incident: after a success has been accepted, a failed event with a later timestamp arrives. Demonstrate durable success precedence, explain why a timestamp-only reducer is wrong, and show the worker-restart/reconciliation test. The provider scaffold is supplied so this does not become a new queue-system implementation.

`database/performance.sql` creates a separate 50,000-row query lab. Capture `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` before and after your index/query improvement; explain row estimates, filters, sort/index access and write cost. Preserve identical result IDs. **No absolute millisecond pass threshold.**

Synthetic data dictionary and provenance

All records, amounts and campaign names are invented for this assessment. There are no real customers, contacts, phone numbers, emails, credentials or provider transactions. `seed.json` is canonical; CSVs are human-readable projections of its arrays and preserve repeat delivery rows. Do not infer truth from the starter's defective summary.

Source	Grain / meaning	Caveat
`seed.json:tenants`	two lab tenants `north`, `south`	header-selected simulated auth
`seed.json:campaigns`	one synthetic campaign ID	ID and tenant both matter
`orders.csv` / `orders`	one order, positive integer `amountPaise`, INR, authoritative status snapshot	paid remains paid after a refund; createdAt is not gateway delivery time
`refunds.csv` / `refunds`	one refund ledger record	only `settled` subtracts net; pending does not; full refund still counts a paid order
`ad-metrics.csv` / `adMetrics`	one tenant/campaign/Asia-Kolkata calendar day	spendPaise authoritative for spend; reportedConversions and reportedRevenuePaise are attributed estimates, not payment truth
`gateway-deliveries.csv` / `paymentEvents`	one raw delivered event attempt	exact repeated event `e-n1`, new event ID for same business payment, late failure and offset/date boundaries

Source	Grain / meaning	Caveat
`seed.json:payments`	one current `(tenant,paymentId)` projection	succeeded dominates later failed/pending snapshots

Loading and reconciliation

`node server/reset.mjs` recreates only the guarded assessment schema and reads `seed.json` using parameterized inserts. Eight raw gateway delivery rows become seven canonical `payment_events` because the exact event ID duplicate is collapsed. Distinct event IDs for the same payment remain separate observations. Five payment projections and eight orders remain independent grains. Runtime outbox/audit/provider/fulfilment tables intentionally start empty: historical paid orders are already serviced and are not backfilled.

All money is **integer paise** (100 paise = INR 1); no float amount input. Count paid orders once, including fully refunded paid orders. Gross revenue is the sum of paid-order amounts; net revenue subtracts settled refunds linked to those orders. Pending refunds do not reduce net. Sum ad spend independently at its source grain. Net ROAS is net revenue divided by spend, or null when spend is zero. Derive each tenant's totals from the fixtures and include reproducible SQL evidence.

Dates: `2026-08-02T00:05:00+05:30` is August 1 UTC but August 2 in the business calendar. `occurredAt` is provider observation time, `received_at` database ingestion time, and `createdAt` order creation time; do not interchange them. Ordering a status snapshot by timestamp cannot override the documented success precedence. Summary has no date filter; the timezone task concerns diagnosing daily exports.

Suggested diagnosis: compare reported conversions/revenue to distinct paid orders, account for retries and unsettled/settled refunds, state the time-zone convention and identify why campaign attribution is not accounting. Use SQL evidence rather than assuming the current API display is correct. `server/API_CONTRACT.md` governs all endpoint/business behavior. `database/performance.sql` is deliberately separate from this small financial fixture.